4 ТЕХНІКИ ПРОЕКТУВАННЯ ДИНАМІЧНИХ ТЕСТОВИХ ВИПАДКІВ

4.1 Вступ

Тестування програмного забезпечення (ПЗ) передбачає перевірку відповідності ПЗ специфікації. Для перевірки використовують тести, або тестові випадки, які можуть бути спроектовані для динамічної та статичної перевірки.

Метою цієї лекції є ознайомлення з техніками (методами) проектування динамічних тестових випадків, тобто тих, що вимагають виконання тестованої програмної сутності з певними наборами значень вхідних даних. У ході лекції буде розглянуто методи тестування білої та чорної скриньки.

4.2 Різновиди технік (методів) розробки тестових випадків

Техніки розробки тестових випадків поділяють на методи розробки статичних та динамічних тестів.

Техніки розробки статичних тестів передбачають тестування без виконання коду чи програмного додатку. Таке тестування в основному пов'язане із забезпеченням якості, включаючи перевірку та аудит коду та інших проектних документів.

Техніки розробки динамічних тестів передбачають тестування шляхом запуску тестованої системи. У цих техніках тестувальник надає вхідні дані до програми та виконує її. Це робиться для перевірки його різних функціональних і нефункціональних вимог.

Методи розробки динамічних тестів можуть бути на основі:

- структури (біла скринька);
- специфікацій (чорна скринька);

досвіду – не вимагають жодного систематичного та вичерпного тестування. Вони повністю базуються на досвіді чи інтуїції тестувальника.

При тестуванні білою скринькою є доступ до вихідного коду програми, тому його називають тестуванням, що керується логікою. Його обов'язковими атрибутами є повна та точна специфікація, а також тести/тестові випадки – набори вхідних і вихідних даних (мінімальні допустима кількість полів).

При тестуванні білою скринькою хорошим є тест, який має високу ймовірність виявлення помилок.

Наведемо кілька прикладів специфікацій функцій, які можуть бути використанні для тестування білою скринькою.

```
/// <summary>
/// пошук першого максимального елементу масиву
/// </summary>
/// <param name="arr">масив цілих чисел</param>
/// <param name="size">кількість елементів у масиві</param>
/// <returns>-1 якщо масив порожній, 0+ позиція першого максимального елементу</returns>
/// <summary>
/// перевірка не порожнього рядка на наявність цифр
/// </summary>
/// <param name="text">рядок для перевірки</param>
/// <returns>чи містить рядок цифри</returns>
```

Такі специфікації можуть бути доповнені інформацією про типи даних вхідних і вхідних значень.

4.3 Методи тестування білої скриньки

До методів білої скриньки відносяться:

- покриття операторів;
- покриття рішень;
- покриття умов;
- покриття умов та рішень;
- комбінаторне покриття умов.

Метод покриття операторів передбачає підбір таких наборів вхідних даних, при яких кожен оператор програми виконається хоча б один раз. Результати тестування можуть бути оформлені у вигляді таблиці з вказівкою номерів операторів (присвоєні за порядком заздалегідь) по вертикалі та номерів тестів по горизонталі.

Розглянемо один з можливих варіантів оформлення проектування тестів цим методом (табл. 1). Відмітка про виконання оператору ставиться на перетині відповідного рядка та стовпця, що відповідає тесту, який виконується.

```
int NumberMax(int a, int b){
if (a>b) //1
        return 1; //2
else
if (b>a) //3
                return 2; //4
return 0; //5
}
```

T1. Вхід: a=0, b=0
Вихід: 0
T2. Вхід: a=1, b=0
Вихід: 1
T3. Вхід: a=1, b=5
Вихід: 2

Таблиця 1

Покриття операторів тестами

Номер оператора / Ідентифікатор тесту	1	2	3	4	5
T1	*		*		*
T2	*	*			
T3	*		*	*	

При тестуванні під рішенням будемо розуміти набір умов та їх значень, що визначають хід виконання програми. До набору умов слід включити умови всіх керуючих структур: циклів усіх видів (for, while, do-while для мови 5 C/C++) та розгалужень (if, switch). Приклади умов на мові C: a>b, current==true, a>b || a!=0, b==4. При покритті рішень кожен напрям (кожен можливий варіант з набору

умов та значень) має бути реалізовано хоча б один раз. Приклад оформлення покриття рішень наведено в табл. 2.

Таблиця 2

Приклад позначення покриття рішень

№ тесту	Рішення 1		Рішення 2		Рішення 3	
	+	-	+	-	+	-
1	*		*		*	
2		*		*		
3	*		*	*		*

Метод покриття умов передбачає створення такого набору тестів, за яким кожна умова виконається та не виконається хоча б один раз. Якщо є складені умови, то кожна частина тестується окремо. Результати можуть бути представлені аналогічно методу покриття операторів. При тестуванні методом покриття рішень та умов можливі результати кожної умови у рішенні отримані принаймні один раз і всі рішення виконались хоча б один раз. У методі комбінаторного покриття умов необхідно протестувати всі можливі комбінації умов.

4.4 Методи тестування чорною скринькою

Тестування чорною скринькою – це тестування, що керується даними, тому доступ до вихідного коду програми відсутній. Наявність повної та точної специфікації є обов'язковою необхідною умовою для проведення такого тестування. Мінімальна кількість даних, що вказується у тестах, – це набори значень вхідних і вихідних даних.

Методи чорної скриньки:

- еквівалентне розбиття;
- аналіз граничних значень;
- припущення про помилку;
- функціональні діаграми;
- перехід за станами.

У методі еквівалентного розбиття множина значень вхідних даних функції, що утворюють її вхідний простір, розбивається на набір підмножин таким чином, що в кожну підмножину потрапляють значення, еквівалентні одне одному з точки зору їх використання в тестах для виявлення помилок. Набори даних такого класу об'єднують за принципом виявлення одних і тих же помилок: якщо набір будь-якого класу виявляє деяку помилку, то передбачається, що всі інші тести цього класу еквівалентності теж виявлять цю помилку і навпаки. Усі тести, які можуть бути побудовані на основі еквівалентних значень, представляють один «клас еквівалентності», і для тестування досить вибрати тільки один тест з класу.

Виділення класів є евристичним процесом, проте рекомендується виділяти в окремі класи допустимі та недопустимі значення деякого параметру. Також слід брати до уваги такі випадки:

- якщо значення параметру задається діапазоном, доречно розглянути три класи: один – у межах, два з права і зліва поза межами;

- якщо специфікація описує множину вхідних значень і є підстави вважати, що кожне значення трактується особливо, то слід виділити клас під кожне значення, і один – неправильний, який не входить до множини;
- якщо задано умову «має бути», то слід розглянути два класи, один з яких задовольняє умову, інший – ні.

У методі аналізу граничних значень, що доповнює попередній метод, дані вибираються на границях вхідної області, оскільки багато відмов виникає через дефекти, пов'язані з обробкою граничних значень вхідних параметрів.

Розглянемо приклад застосування методу аналізу граничних значень (умов).

Функція визначає відсоткову ставку депозиту в залежності від терміну вкладу. Для 2 років 10%, від двох до п'яти років включно 20%, понад п'ять років 30%. Для строку менше 2 років 0 %. Термін вкладу – ціле число.

У табл. 3 наведено тестові дані для методу граничних умов. Вони передбачають перевірку границь та значень праворуч та ліворуч від них.

Таблиця 3

Тести, розроблені за методом аналізу граничних значень

Тест	Вхід	Вихід	Отриманий результат
1	2	10	Буде отримано в ході виконання програми
2	5	20	
3	1	0	
4	3	20	
5	6	30	

Тестування методом припущення про помилку – це процес, що має в значній мірі інтуїтивний характер, вимагає врахування конкретних обставин. Основна ідея: скласти список можливих помилок або ситуацій, в яких вони проявляються, а потім написати під них тести.

Для прикладу візьмемо функцію сортування масиву за зростанням методом бульбашки. Тоді припущення мають стосуватися таких варіацій вхідних даних:

- масив порожній;
- масив має лише один елемент;
- масив має елементи рівні за значенням;
- масив вже впорядкований за зростанням.

За методом функціональних діаграм спочатку ідентифікується кожна окрема функція, потім визначаються всі причини, які впливають на її резуль□тат, і всі відповідні реакції (наслідки). Наступний крок полягає в побудові графа (діаграми), що зв'язує комбінації причин з очікуваними реакціями на них.

Діаграма може супроводжуватися примітками щодо обмежень та комбінацій на вхідні умови. Далі для кожного наслідку, зазначеного на графі, визначаються тестові набори шляхом перебору всіх комбінацій причин, що породжують цей наслідок.

На діаграмі для позначення умов та їх комбінацій використовують такі позначки: – пряма лінія – тотожність (рис. 1 а, причина «п1» породжує наслідок «н3»); – тильда «~» – заперечення (рис. 1 б); – кон'юнкція «^» – «і» (рис. 1 б,

якщо є причини «п3» і відсутня «п1», маємо наслідок «н2»); – диз'юнкція «∨» та дуга – «або» (рис. 1 в, якщо є причина «п1» або «п2» ма□ємо наслідок «н1»).

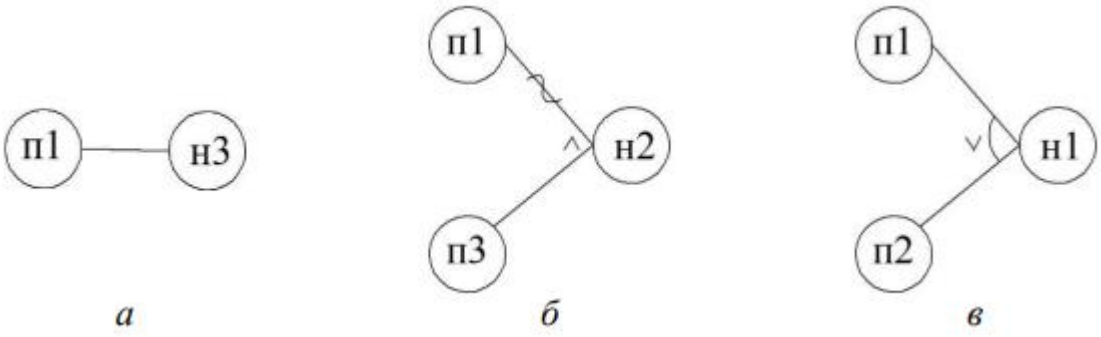


Рис. 1. Умовні позначення для діаграми:
а – тотожність; б – заперечення та «і»; в – «або

Також є додаткові позначення, які позначаються пунктирною лінією та буквою:

- E – взаємне виключення причин;
- I – присутня хоча б одна з причин;
- O – можлива одна і лише одна з поєднаних причин;
- R – одна причина вимагає наявність іншої;
- M – приховування (якщо причина приймає значення одиниці, то поєднаний наслідок – нуль).

Розглянемо приклад. Функція визначає номер максимального з двох заданих чисел. Нуль – ознака рівності чисел.

Вхід: a – перше ціле число, b – друге ціле число.

Вихід: 0 – числа рівні, 1 – перше число більше за друге, 2 – друге число більше за перше.

Сформуємо причини і наслідки для описаної функції (табл. 4).

Таблиця 4

Опис причин та наслідків

Ідентифікатор причини(П)/наслідку(Н)	Опис причини/наслідку	Помітки (комбінації причин та наслідків)		
		1	2	3
П1	Числа рівні	*		
П2	перше число більше за друге		*	
П3	друге число більше за перше			*
Н1	Максимальне число не можливо визначити – 0	*		
Н2	Номер максимального числа дорівнює 1		*	
Н3	Номер максимального числа дорівнює 2			*

Функціональна діаграма, що відповідає табл. 4, наведена на рис.

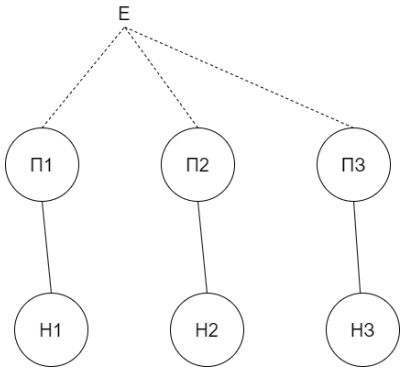


Рис. 2. Функціональна діаграма

Тести для помiток, наведених у табл. 4:

1 – Т1. Вхід: a=0, b=0

Вихід: 0

2 – Т2. Вхід: a=1, b=0

Вихід: 1

3 – Т3. Вхід: a=1, b=5

Вихід: 2

При тестуванні методом переходів за станами слід дотримуватися такого порядку дій:

- виділити стани системи,
- визначити умови та послідовність переходів між станами,
- підібрати вхідні дані таким чином, щоб спрацював кожний перехід.

Стани можуть бути представлені графічно. Приклад представлення станів для функції логування в банкоматі наведено на рис. 3.

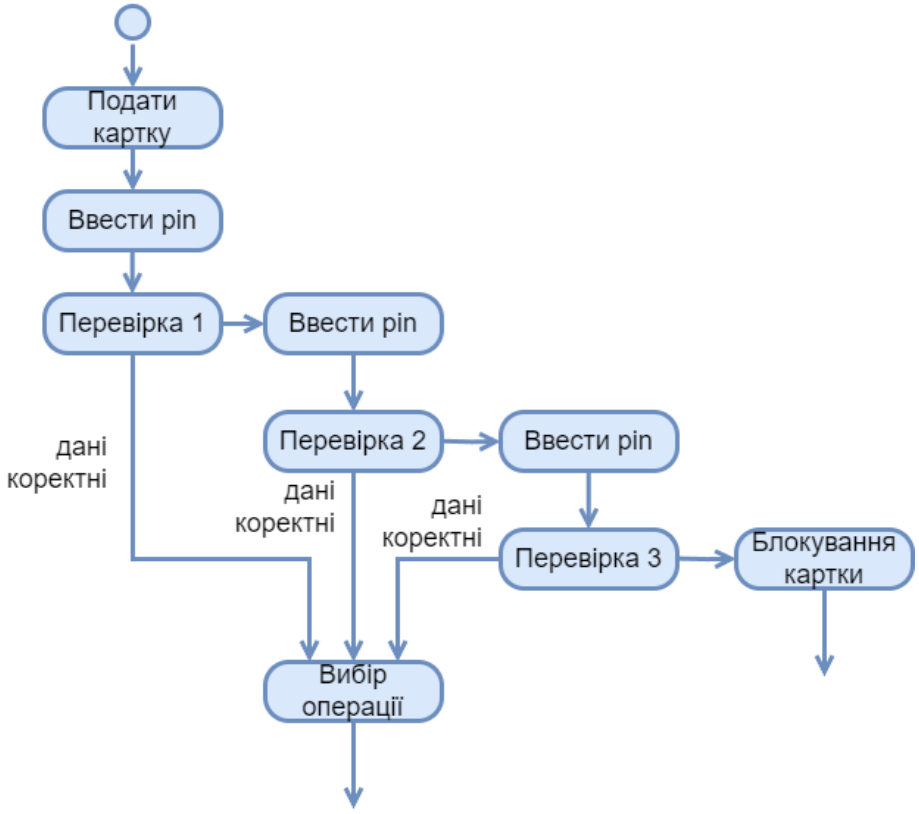


Рис. 3. Фрагмент діаграми станів

Далі для даного прикладу варто спроектувати тести, які покриють усі випадки коректних та некоректних даних.

4.5 Висновки

Підведемо підсумки цієї лекції:

- застосування динамічних тестів передбачає виконання програми;
- тести можуть проектуватися на основі структури, специфікації, досвіду;
- при проектуванні на основі структури застосовують методи білої скриньки;
- при проектуванні на основі специфікації застосовують методи чорної скриньки;
- кращим варіантом буде поєднання кількох технік.